Case Study: Knowledge-Engine-as-a-Service (KaaS) Platform

1. Executive Summary

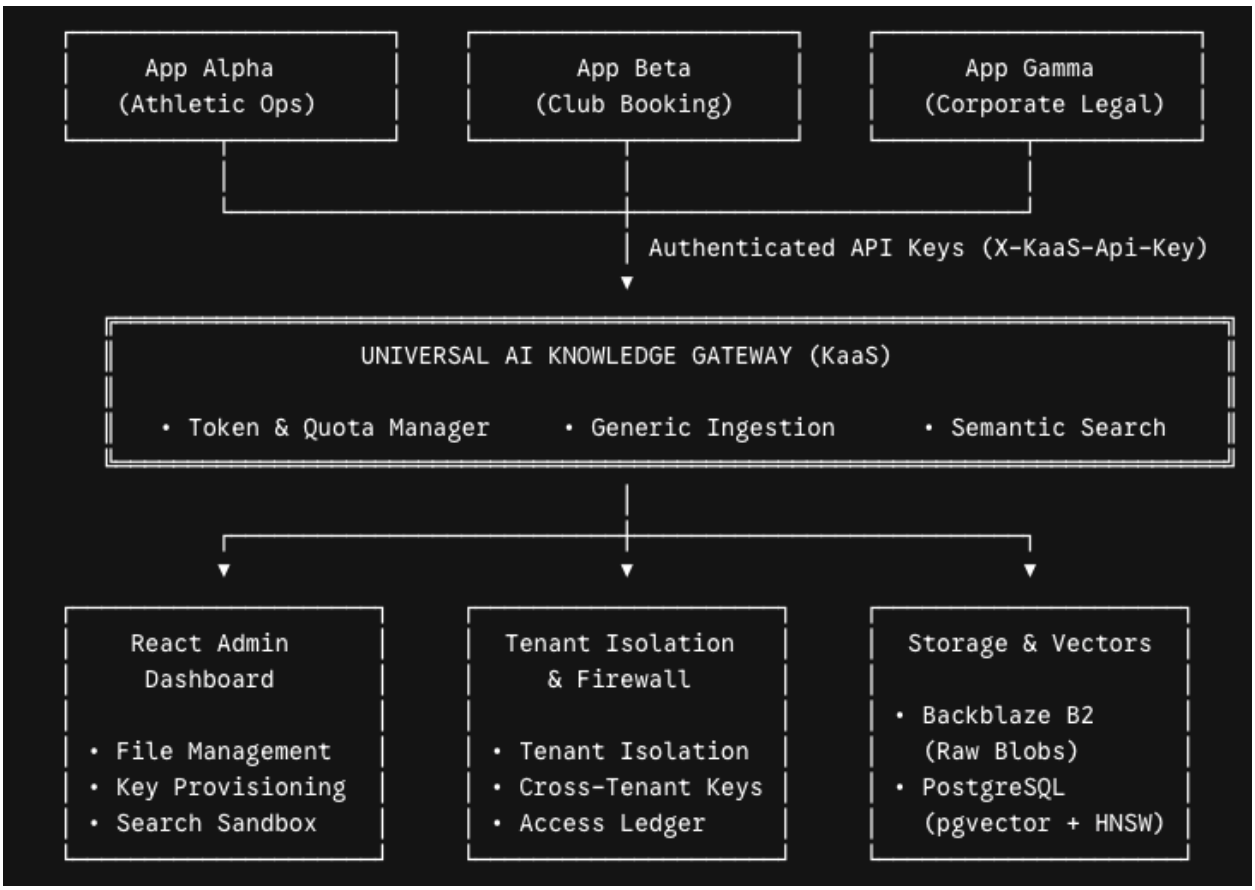
This project transforms an isolated, application-specific search feature into a standalone **Knowledge-Engine-as-a-Service (KaaS)** platform.
Instead of forcing individual software applications to build custom document loaders, chunking logic, and vector stores, the KaaS platform acts as a universal, private AI retrieval gateway—comparable to a self-hosted, enterprise-controlled alternative to Custom GPTs or Dify. Any authorized application can stream raw documents into the platform via standard APIs and instantly query them with secure, context-aware semantic search.

2. The Problem vs. The Solution

Challenge with Isolated RAG	The KaaS Solution
Siloed Development: Every app writes duplicate document parsing, chunking, and embedding logic.	**Single Gateway:** A unified API handles file parsing, vectorization, and retrieval across all applications.
Domain-Locked Data Models: Schemas hardcoded to specific entities (e.g., tennis courts, legal files) fail when scaled.	**Abstract Payloads:** Generic metadata envelopes allow any business vertical to reuse the same backend engine.
Zero Data Mobility: Multi-tenancy strictly isolates data; systems cannot share or monetize valuable documentation.	**Access Ledger Pattern:** Granular permission rules support private tenant isolation, cross-organization sharing, and commercial knowledge licensing.
Uncontrolled Token Usage: Apps lack centralized budget caps, leading to unpredictable LLM and vector compute costs.	**Centralized Budgeting:** API key validation enforces hard rate limits and monthly usage caps per tenant.

3. System Architecture & Workflow

External applications interact exclusively with the KaaS Gateway through authenticated API keys. The gateway verifies permissions, routes files to object storage, and manages semantic vector queries in PostgreSQL.



4. Key Architectural Pillars

Pillar 1: The Multi-Tenant Access Ledger

Traditional multi-tenant systems enforce isolation by matching a single tenant ID:

```
SQL
WHERE TenantId == @CurrentTenantId
```

While secure, this prevents organizations from collaborating or licensing their data. KaaS introduces a dual-layer **Access Ledger Pattern**:

- **Private by Default:** Documents remain restricted to the owner tenant unless explicitly changed.
- **Granular Sharing:** An IsPubliclyShared flag marks public libraries, while a TenantAccessGrant lookup table permits controlled access to specific third-party tenants.
- **Commercialization:** Premium operators can license specialized operational procedures, guidelines, or training content to partner companies for a subscription fee without losing custody of their files.

```C#
// Universal Multi-Tenant Access Filter
var accessibleRecords = await _context.KnowledgeRecords
    .Where(k => k.TenantId == userTenantId
            || k.IsPubliclyShared
            || _context.TenantAccessGrants.Any(g => g.RecordId == k.Id && g.GranteeTenantId
== userTenantId))
    .OrderBy(k => k.Embedding.CosineDistance(queryVector))
    .Take(topK)
    .ToListAsync();
```

Pillar 2: Domain-Agnostic Web API & Token Budgeting

To support multiple industries without redesigning the schema, the ingestion engine decouples from domain-specific properties:

- **Generic Payloads:** Instead of binding to fixed entities like CourtId or CaseFileId, endpoints accept an abstract AssetReferenceKey along with standard key-value metadata tags.
- **Token Budgeting & Security:** Callers authenticate with scoped headers (X-KaaS-Api-Key: k_live_...). A lightweight Redis middleware validates the key, checks current consumption against monthly budget thresholds, and throttles excess requests before vector operations run.
- **Asynchronous Processing:** Documents upload directly to encrypted object storage (Backblaze B2), triggering background workers to parse text, compute embeddings, and build vector indexes.

Pillar 3: Web-Based Management Console

A unified React interface gives non-technical operators and system administrators full control over their knowledge bases:

- **Document Ledger:** Real-time visibility into uploaded assets, indexing status (Queued, Processing, Indexed, Failed), storage footprint, and privacy levels (Private, Shared, Public).
- **RAG Playground:** An interactive test sandbox allowing administrators to run sample queries, inspect matched document chunks, and review similarity scores before client applications consume the index.

5. Security & Isolation Matrix

Layer	Technical Implementation	Operational Safeguard
Row-Level Tenancy	EF Core Global Query Filters + PostgreSQL RLS	Prevents data leakage across tenant boundaries.

Storage at Rest	AES-256 encrypted object storage partitioned by Tenant ID	Isolates raw source files physically and cryptographically.
Transit Security	Strict TLS 1.3 across all REST and vector streaming routes	Protects API payloads and retrieval streams against interception.
API Control	Scoped tokens with instant Redis cache invalidation	Enables immediate revocation of compromised keys.
Compliance Logging	Append-only audit logs for all uploads, queries, and grants	Meets governance standards for SOC 2, HIPAA, and GDPR auditability.

6. Implementation Roadmap

Plaintext

Phase 1: Foundation (Weeks 1-4)
• Set up PostgreSQL schema with pgvector extensions and tenant isolation filters.
• Implement the C# Universal Ingress Controller and document streaming pipeline.
• Configure encrypted Backblaze B2 object storage integration.

Phase 2: Federation & Token Budgeting (Weeks 5-8)
• Deploy Redis-backed rate limiting and monthly tenant token budgeting.
• Implement the Access Ledger system (IsPubliclyShared and TenantAccessGrant).
• Publish generic REST API endpoints and client authentication libraries.

Phase 3: Administrative Dashboard & Launch (Weeks 9-12)
• Build the React management console for document tracking and key provisioning.
• Deliver the embedded RAG Sandbox for live query testing and tuning.
• Complete end-to-end security audits, load testing, and production deployment.

7. Technology Stack and Tooling

CORE TECHNOLOGY STACK	
API & Application Framework	• C# / .NET 9 (ASP.NET Core Web API) • Entity Framework Core (EF Core 9) • Minimal APIs & FastEndpoints
AI Orchestration & RAG Pipeline	• Microsoft Semantic Kernel (Agent Framework) • Microsoft.Extensions.AI abstractions • Dense Embedding Models (text-embedding-3) • Semantic Header-Aware Chunker
Database & Vector Engine	• PostgreSQL 16+ with pgvector extension • HNSW (Hierarchical Navigable Small World) • Npgsql.EntityFrameworkCore.PostgreSQL
Storage & Caching	• Backblaze B2 (S3-compatible, AES-256 at rest) • Redis (StackExchange.Redis) for token quotas
Ingestion & Queue	• Background Hosted Services (.NET Channels) • MassTransit / RabbitMQ for async processing
Administrative Web Console	• React 18+ with TypeScript • Tailwind CSS & Shadcn UI • Vite build tooling
Observability & Containerization	• OpenTelemetry (.NET OTel SDK) • Docker & Docker Compose • Prometheus & Grafana